Gate 1: entrega y verificación de la miniaplicación (Bloque 11) / Verificación en vivo: clonar, instalar y ejecutar sin ayuda

✓ **Hecho:** Ver

Verificación en vivo: clonar, instalar y ejecutar sin ayuda

Ya sabes exactamente qué se evalúa en el **Gate 1** y bajo qué condiciones se aprueba.

> Esta lección **es un ensayo de la parte más literal de esa evaluación**: alguien que no es tú, sin tu ayuda y sin la memoria de cómo configuraste tu propio computador, tiene que poder clonar tu repositorio, instalarlo y ejecutarlo.

Practicar esto ahora, en tus propias condiciones, es la única forma real de anticiparte a lo que puede fallar antes de que falle frente a quien te evalúa.

> **Prerrequisitos**: No necesitas instalar nada nuevo ni una versión distinta de Python: usa la misma que ya tienes funcionando desde el Bloque 3. Sí necesitas tres cosas antes de empezar: Git instalado y funcionando en tu terminal, tu repositorio ya subido a GitHub (no solo guardado en local: vas a clonarlo desde su URL remota), y unos cinco minutos sin interrupciones, porque el ensayo se hace de punta a punta, sin pausas donde "ya sabes lo que sigue".

🔎 Por qué esto se prueba, y no solo se revisa

Una revisión de tus archivos, por cuidadosa que sea, no detecta todos los problemas de entorno:

⚠️

Una dependencia que instalaste a mano y nunca agregaste a tu lista de dependencias.

⚠️

Una ruta escrita a mano que solo existe en tu computador.

⚠️

Un paso que “ya sabes hacer” y por eso nunca escribiste en el **README**
.

Ese tipo de fallas solo aparece cuando alguien intenta usar tu proyecto de verdad, desde cero, sin ningún conocimiento previo de cómo lo armaste.

Esto es exactamente lo que evalúan, con distinto peso, dos de las seis dimensiones de la rúbrica: "Configuración del entorno y repositorio" (20 %) y "Documentación y versionamiento" (10 %).

✅ **Un ensayo exitoso de esta lección es evidencia directa de ambas**.

⚠️ Por qué tu propio computador no es la prueba perfecta

Hacer este ensayo en tu propia máquina, en una carpeta nueva, es muchísimo mejor que no hacerlo, pero tiene un límite que conviene que conozcas: tu computador ya tiene **Python** instalado, probablemente ya tiene librerías instaladas de forma global en algún momento del programa, y puede que hasta tengas variables de entorno configuradas a nivel de sistema operativo sin acordarte.

Nada de eso existe en el computador de quien va a evaluar tu **Gate**, y por eso tu propio ensayo puede "funcionar" incluso con una falla real, simplemente porque tu máquina la está tapando sin que lo notes.

> 💡 Si quieres una prueba todavía más cercana a la real, la mejor opción es pedirle a un compañero del programa que clone tu repositorio en su propio computador y lo ejecute siguiendo solo tu README, sin que tú lo guíes en tiempo real.

> 💡 Si eso no es posible, otra alternativa razonable es un entorno en la nube que no comparte nada con tu configuración local (por ejemplo, un Codespace o un contenedor limpio), donde ninguna instalación previa tuya puede estar disimulando el problema.

Si ninguna de esas dos opciones está a tu alcance, el ensayo en tu propia máquina, en una carpeta nueva y con un entorno virtual nuevo, sigue siendo muchísimo mejor que no ensayar nada. Solo ten en cuenta que es una aproximación, no una garantía absoluta.

🧪 El ensayo, paso a paso

El ensayo consiste en repetir, en una carpeta completamente nueva de tu propio computador, exactamente lo que haría alguien que nunca vio tu proyecto:

1. 📂 **Clona tu propio repositorio en una carpeta nueva y vacía,** distinta de la que has estado usando para desarrollar. Si algo de tu configuración vive solo en tu carpeta de trabajo actual y no en el repositorio, esta es la forma de descubrirlo.

2. 🧪 **Crea un entorno virtual nuevo, desde cero,** y actívalo, con el mismo procedimiento que ya usaste en el **Bloque 3**, según tu sistema operativo.

3. 📦 **Instala las dependencias usando solo lo que está declarado en tu repositorio.** Nada instalado "**a mano**" fuera de tu lista de dependencias cuenta para este ensayo.

4. 🔐 **Crea tu archivo ".env" a partir de tu propio ".env.example",** y completa ahí tu **API key**. Si no tienes un "**.env.example**" en tu repositorio, esa es, en sí misma, una falla que este ensayo te permite descubrir antes de la entrega.

5. ▶️ **Ejecuta el proyecto siguiendo únicamente las instrucciones de tu propio README,** sin usar ningún atajo, alias o configuración que solo tú, por experiencia, sabrías aplicar.

```
# macOS / Linux / WSL
git clone <url-de-tu-repositorio> prueba-gate1
cd prueba-gate1
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python3 main.py  # o el nombre real de tu script principal

# Windows (PowerShell)
git clone <url-de-tu-repositorio> prueba-gate1
cd prueba-gate1
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
python main.py  # o el nombre real de tu script principal
```

Completa tu API key dentro de tu nuevo ".env" antes del último paso: la copia (ya sea "cp" o "Copy-Item") solo trae la plantilla vacía, no tu credencial real. Nota también la distinción entre "python3" (macOS, Linux y WSL) y "python" (Windows): es la misma que ya usaste en el Bloque 3, no algo nuevo de este ensayo.

No hay una salida única correcta, porque depende de qué imprime tu propio script principal, pero la señal de un ensayo exitoso se ve, en términos generales, así:

```
(venv) $ python3 main.py
Respuesta del modelo: ... # el contenido real de tu miniaplicación
Proceso terminado sin errores.
```

Lo que importa no es el texto exacto, sino dos cosas: que el programa corra hasta el final sin que aparezca un "Traceback" en rojo, y que lo que imprime sea el resultado real de tu miniaplicación (una respuesta del modelo, un registro guardado, lo que corresponda), no un mensaje de error disfrazado de salida normal.

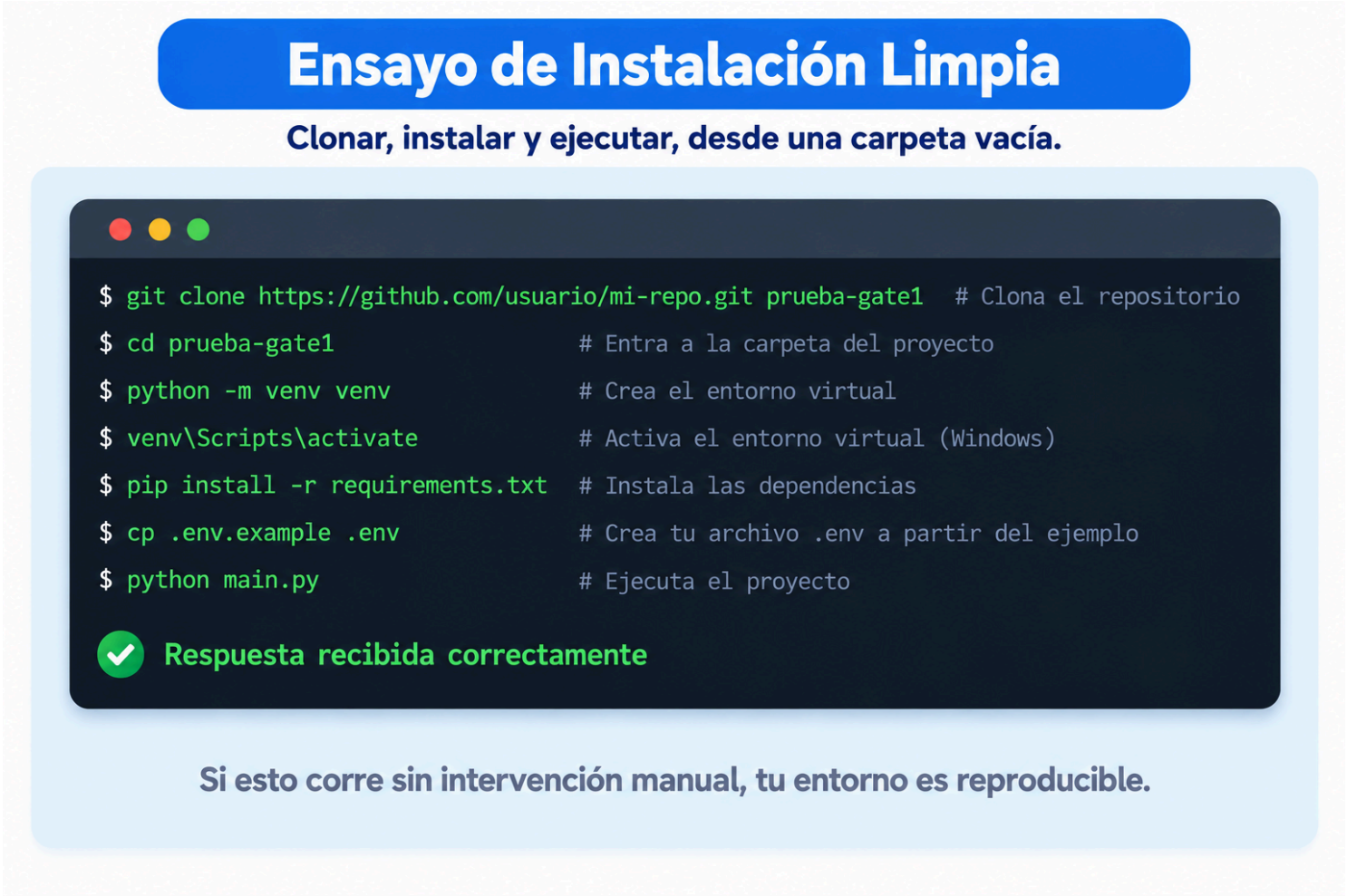


Verificación de seguridad

Antes de seguir, en esta misma copia clonada (no en tu carpeta de trabajo original), corre **"git status"** y confirma que tu archivo **".env"** no aparece listado. Si tu **".gitignore"** está funcionando como corresponde, no debería aparecer, porque **"git clone"** trae exactamente lo que hay en tu repositorio remoto, y tu **".env"** real nunca debería haber llegado ahí.

🔐 **Si aparece:** tu credencial ya quedó expuesta en el historial. La explicación completa de cómo verificar esto y qué significa si falla está en la Unidad 1.2, documento **"Variables de entorno: cómo excluir tus secretos del repo"**.

Si activar el entorno virtual te da problemas según tu sistema operativo, revisa la guía de la **Lección 5** del **Bloque 3** ➡️ **Windows, macOS o WSL: la guía para cuando algo falla** : las diferencias que viste ahí son exactamente las que pueden aparecer en este ensayo.



⚠️ Qué falla más seguido en este ensayo

Falta "requirements.txt", o está desactualizado. Instalaste una librería a mano en

algún momento del desarrollo y nunca quedó registrada.

No hay ".env.example", o el README no explica qué variables se necesitan. Quien clona tu repositorio no tiene forma de adivinar el nombre exacto de la variable de entorno que tu código espera.

El código asume una ruta o un archivo que solo existe en tu computador. Una ruta absoluta escrita a mano, en vez de una ruta relativa al proyecto, es la causa más común de este problema.

El README da por hecho pasos que para ti son obvios, pero no lo son para alguien externo. "Instala las dependencias" sin decir con qué comando, o "**configura tu API key**" sin decir dónde, no es instrucción suficiente para este ensayo.

El ".gitignore" está incompleto, y algo que no debería subirse quedó en el repositorio. Es la falla opuesta a las anteriores, y la más delicada: revisa tu propio repositorio clonado (no tu carpeta de trabajo original) y confirma que no aparece ni tu carpeta de entorno virtual ni, sobre todo, tu archivo "**.env**" con tu **API** key real. Si aparece, tu credencial ya quedó expuesta en el historial de **commits**, y no basta con borrarla del último **commit** para solucionarlo.

Deja un registro de tu ensayo

No te quedes solo con **"lo intenté y funcionó"**: anota, aunque sea en un par de líneas dentro de tu **README** o en un archivo aparte, qué encontraste y qué corregiste.

Ese registro es, en sí mismo, evidencia de que hiciste el ensayo en serio, y es exactamente el tipo de rastro que la dimensión "Documentación y versionamiento" de la rúbrica espera encontrar.

```
## Registro del ensayo de instalación limpia

- Fecha: 2026-09-08
- Encontrado: faltaba 'requests' en requirements.txt (estaba instalado a mano)
- Corregido: agregado con 'pip freeze > requirements.txt' y commiteado
- Resultado tras la corrección: el proyecto corre sin intervención manual
```

Si tu primer ensayo ya sale limpio, sin ninguna falla, igual **vale la pena dejar constancia de que lo hiciste**: una línea que diga "ensayo de instalación limpia realizado el [fecha], sin hallazgos" es evidencia tan válida como el registro de una corrección.

🛠️ Si algo falla en tu ensayo

Encontrar una falla en este ensayo no es un mal resultado: es exactamente el propósito de la lección, y es muchísimo mejor descubrirla ahora que durante la entrega real de tu **Gate 1**.

Corrige lo que corresponda (agrega la dependencia faltante, crea o completa tu **".env.example"**, ajusta el **README**, reemplaza la ruta absoluta por una relativa), haz un commit del cambio, y repite el ensayo completo hasta que corra sin que tuvieras que intervenir manualmente en nada que no esté documentado.



Siguiente actividad

Gate 1 · Entrega formal Actividad Core

Con tu proyecto ya verificado de esta forma, en la última lección de esta unidad vas a hacer la entrega formal del **Gate 1**: reunir todas las piezas del módulo, confirmar que cumplen los criterios no compensables y el umbral de la **rúbrica**, y presentarlas para su evaluación.

Última modificación: jueves, 24 de septiembre de 2026, 17:48